

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

QUESTION 1 — Star Topology

The question requires a Python program that accepts the number of hosts, creates a star topology with a central switch, displays each connection and Cat6 UTP cable length, and verifies connectivity.

Algorithm

1. Read the number of hosts.
2. Create a central switch.
3. Create Host1 to HostN.
4. Assign a Cat6 cable length to each host.
5. Display every host-to-switch connection.
6. Verify that every host is connected to the switch.
7. Display the topology in ASCII format.

Python Program

```
def create_star_topology(num_hosts):
    if num_hosts <= 0:
        print("Error: Number of hosts must be greater than 0.")
        return

    switch = "Central Switch"
    connections = []

    print("\nSTAR TOPOLOGY")
    print("=" * 45)
    print(f"          [{switch}]]")
    print("          |")

    for i in range(1, num_hosts + 1):
        host = f"Host{i}"

        # Example Cat6 cable length
        cable_length = 10 + (i * 5)

        connections.append((host, switch, cable_length))

    print(f"          |")
    print(f"          {host}")

    print("\nCONNECTION DETAILS")
    print("-" * 45)

    for host, switch, length in connections:
        print(
            f"{host} --> {switch} | "
            f"Cat6 UTP Cable = {length} meters"
        )

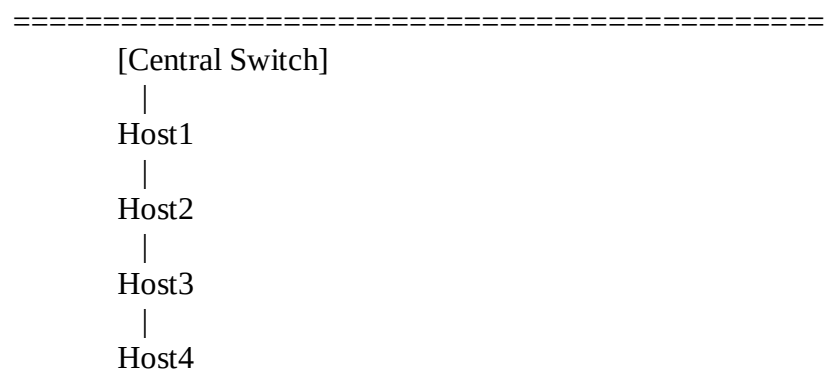
    # Connectivity verification
    if len(connections) == num_hosts:
        print("\nConnectivity Verification: PASSED")
        print(f"All {num_hosts} hosts are connected through {switch}.")
    else:
        print("\nConnectivity Verification: FAILED")
```

```
# Main program
try:
    n = int(input("Enter number of hosts: "))
    create_star_topology(n)
except ValueError:
    print("Error: Please enter a valid integer.")
```

Sample Output

Enter number of hosts: 4

STAR TOPOLOGY



CONNECTION DETAILS

```
-----
Host1 --> Central Switch | Cat6 UTP Cable = 15 meters
Host2 --> Central Switch | Cat6 UTP Cable = 20 meters
Host3 --> Central Switch | Cat6 UTP Cable = 25 meters
Host4 --> Central Switch | Cat6 UTP Cable = 30 meters
```

Connectivity Verification: PASSED

All 4 hosts are connected through Central Switch.

Explanation

In a star topology, every host has a separate connection to the central switch. If one cable fails, the other hosts can continue communicating. The central switch provides a simple and reliable method of managing LAN communication.

QUESTION 2 — UTP Cable Validation

The question specifically requires `validate_segment(length_m)`, a maximum permissible length of **100 meters**, appropriate error handling, and at least three test cases.

Algorithm

1. Accept cable length.
2. Check whether the value is numeric.
3. Check whether length is positive.
4. Check whether length is ≤ 100 m.
5. Return True for valid lengths.
6. Return False with an error message for invalid lengths.
7. Test using at least three values.

Python Program

```
def validate_segment(length_m):
    """
    Validates a UTP cable segment according to
    the 100-meter Ethernet cable limit.
    """

    if not isinstance(length_m, (int, float)):
        return False, "Error: Cable length must be a number."

    if length_m <= 0:
        return False, "Error: Cable length must be greater than 0 meters."

    if length_m > 100:
        return False, "Error: Cable length exceeds the 100-meter limit."

    return True, "Valid: Cable segment satisfies the 100-meter limit."

# Test cases
test_cases = [50, 100, 120]

print("UTP CABLE VALIDATION")
print("=" * 45)

for length in test_cases:
    result, message = validate_segment(length)

    print(f"\nCable Length: {length} meters")
    print(f"Result: {result}")
    print(message)
```

Sample Output

UTP CABLE VALIDATION

=====

Cable Length: 50 meters

Result: True

Valid: Cable segment satisfies the 100-meter limit.

Cable Length: 100 meters

Result: True

Valid: Cable segment satisfies the 100-meter limit.

Cable Length: 120 meters

Result: False

Error: Cable length exceeds the 100-meter limit.

Interpretation

- 50 m → Valid
- 100 m → Valid
- 120 m → Invalid

The program correctly accepts cable segments within the specified limit and rejects lengths exceeding it.

QUESTION 3 — CSMA/CD Simulation

The question requires simulation of **four hosts connected through a central switch**, frame transmission, collision detection, random back-off, retransmission, and logging to csma_log.txt.

Algorithm

1. Create four hosts.
2. Generate transmission attempts.
3. Detect simultaneous transmission attempts.
4. Record a collision.
5. Generate a random back-off interval.
6. Wait for the back-off period.
7. Retry transmission.
8. Record successful transmissions.
9. Store all events in csma_log.txt.

Python Program

```
import random
import time
```

```
HOSTS = ["Host1", "Host2", "Host3", "Host4"]
```

```
def log_event(message):
    with open("csma_log.txt", "a") as file:
        file.write(message + "\n")
```

```
def transmit_frames():
    print("CSMA/CD SIMULATION")
    print("=" * 50)

    # Clear previous log
    with open("csma_log.txt", "w") as file:
        file.write("CSMA/CD TRANSMISSION LOG\n")
        file.write("=" * 50 + "\n")
```

```
pending_hosts = HOSTS.copy()
attempt = 1
```

```

while pending_hosts:

    print(f"\nTransmission Attempt {attempt}")
    log_event(f"\nTransmission Attempt {attempt}")

    # Randomly select transmitting hosts
    transmitters = [
        host for host in pending_hosts
        if random.choice([True, False])
    ]

    if len(transmitters) == 0:
        transmitters = [random.choice(pending_hosts)]

    print("Transmitting:", " ".join(transmitters))
    log_event("Transmitting: " + " ".join(transmitters))

    # Collision detection
    if len(transmitters) > 1:

        print("COLLISION DETECTED!")
        log_event("COLLISION DETECTED!")

        for host in transmitters:
            backoff = random.randint(1, 5)

            print(
                f"{host}: Back-off interval = "
                f"{backoff} seconds"
            )

            log_event(
                f"{host}: Back-off interval = "
                f"{backoff} seconds"
            )

            time.sleep(1)

    else:
        successful_host = transmitters[0]

        print(
            f"Successful transmission by "
            f"{successful_host}"
        )

        log_event(
            f"Successful transmission by "
            f"{successful_host}"
        )

    pending_hosts.remove(successful_host)

```



```
attempt += 1
```

```
print("\nAll frames transmitted successfully.")  
log_event("\nAll frames transmitted successfully.")
```

```
transmit_frames()
```

Sample Log

CSMA/CD TRANSMISSION LOG

```
=====
```

Transmission Attempt 1

Transmitting: Host1, Host3

COLLISION DETECTED!

Host1: Back-off interval = 3 seconds

Host3: Back-off interval = 2 seconds

Transmission Attempt 2

Transmitting: Host2

Successful transmission by Host2

Transmission Attempt 3

Transmitting: Host1

Successful transmission by Host1

Explanation

When two or more hosts attempt transmission simultaneously, a collision is detected. The hosts wait for randomly selected back-off intervals before attempting transmission again. This reduces repeated transmission conflicts.

Note for viva: Traditional CSMA/CD is associated with shared, half-duplex Ethernet. Modern switched full-duplex Ethernet normally does not experience collisions.

QUESTION 4 — Client-Server + MAC Address Table

The question requires a Python socket-based client-server application where the server receives frames, forwards them, and maintains a simulated MAC address table.

Server Program

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
mac_table = {}
```

```
def display_mac_table():
```

```
    print("\nMAC ADDRESS TABLE")
```

```
    print("-" * 40)
```

```
    for mac, port in mac_table.items():
```

```
        print(f'MAC: {mac} | Port: {port}')
```

```

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(5)

print("Switch server started...")
print(f"Listening on {HOST}:{PORT}")

while True:
    connection, address = server.accept()

    print(f"\nConnection received from {address}")

    data = connection.recv(1024).decode()

    if not data:
        connection.close()
        continue

    # Expected format:
    # MAC,PORT,FRAME
    parts = data.split(", ", 2)

    if len(parts) == 3:
        source_mac, source_port, frame = parts

        # MAC learning
        mac_table[source_mac] = source_port

        print(f"Received frame: {frame}")
        print(f"Learned MAC: {source_mac}")
        print(f"Associated Port: {source_port}")

        display_mac_table()

        response = "Frame received and MAC address learned."
        connection.send(response.encode())

    else:
        connection.send(
            b"Invalid frame format."
        )

    connection.close()

```

Client Program

```

import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

```

```

source_mac = input("Enter source MAC address: ")
source_port = input("Enter source port: ")
frame = input("Enter frame data: ")

message = f'{source_mac},{source_port},{frame}'

client.send(message.encode())

response = client.recv(1024).decode()

print("\nServer Response:")
print(response)

client.close()

```

Sample Output

MAC ADDRESS TABLE

```

-----
MAC: AA:BB:CC:DD:EE:01 | Port: 1
MAC: AA:BB:CC:DD:EE:02 | Port: 2
MAC: AA:BB:CC:DD:EE:03 | Port: 3

```

Explanation

The switch learns the source MAC address of an incoming frame and associates it with the port where the frame arrived. The MAC address table is then used to determine where frames should be forwarded.

QUESTION 5 — StarSwitch Class

The question requires the StarSwitch class with `add_port()`, `remove_port()`, and `broadcast()` methods, active-port maintenance, broadcast logging, and demonstration with multiple hosts.

Python Program

class StarSwitch:

```

def __init__(self):
    # Dictionary stores host and port information
    self.active_ports = {}

    # Store broadcast history
    self.broadcast_log = []

def add_port(self, host_id):
    """Connect a host to the switch."""

    if host_id in self.active_ports:
        print(f'{host_id} is already connected.')
        return

    port_number = len(self.active_ports) + 1
    self.active_ports[host_id] = port_number

    print(
        f'{host_id} connected to Port {port_number}.'

```

```

    )

def remove_port(self, host_id):
    """Disconnect a host from the switch."""

    if host_id not in self.active_ports:
        print(f'{host_id} is not connected.')
        return

    del self.active_ports[host_id]

    print(f'{host_id} disconnected.')

def broadcast(self, frame, source_host):
    """
    Send a frame to all connected hosts
    except the source host.
    """

    if source_host not in self.active_ports:
        print(
            f'Error: {source_host} is not connected.'
        )
        return

    destinations = [
        host for host in self.active_ports
        if host != source_host
    ]

    print("\nBROADCAST OPERATION")
    print(f'Source: {source_host}')
    print(f'Frame: {frame}')

    if destinations:
        print(
            "Destination Hosts:",
            ", ".join(destinations)
        )
    else:
        print("No destination hosts available.")

    # Store broadcast operation
    self.broadcast_log.append({
        "source": source_host,
        "frame": frame,
        "destinations": destinations
    })

# Demonstration
switch = StarSwitch()

```

```

switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

switch.broadcast(
    "Hello Network",
    "Host1"
)

switch.broadcast(
    "Network Update",
    "Host3"
)

switch.remove_port("Host4")

switch.broadcast(
    "Final Message",
    "Host1"
)

print("\nACTIVE PORTS")
print("-" * 30)

for host, port in switch.active_ports.items():
    print(f'{host} --> Port {port}')

```

Sample Output

```

Host1 connected to Port 1.
Host2 connected to Port 2.
Host3 connected to Port 3.
Host4 connected to Port 4.

```

BROADCAST OPERATION

```

Source: Host1
Frame: Hello Network
Destination Hosts: Host2, Host3, Host4

```

BROADCAST OPERATION

```

Source: Host3
Frame: Network Update
Destination Hosts: Host1, Host2, Host4

```

Host4 disconnected.

BROADCAST OPERATION

```

Source: Host1
Frame: Final Message
Destination Hosts: Host2, Host3

```

Explanation

Broadcasting allows a frame to be sent to all connected hosts except the source host. It is useful

when the destination MAC address is unknown or when network communication needs to reach multiple hosts.